

LAWONCALL PLATFORM

AWS Production Deployment Guide

A comprehensive, step-by-step documentation for deploying the LawMate Full-Stack application (React PWA Frontend, Fastify Microservices Gateway, and PostgreSQL Database) on Amazon Web Services. Written for both technical administrators and non-technical managers.

Status:	Production-Ready / Deployed
Last Updated:	June 10, 2026
Version:	1.1.0 (Detailed Edition)
Author:	Antigravity Dev Team

1. System Architecture Overview

The LawMate (LawOnCall) application is a decoupled, modern multi-service system. It is composed of a single-page PWA client (React/Vite) and six backend microservices (Gateway, Auth, Lead, Payment, Notification, and Profile) coordinated behind an API Gateway proxy. Native AWS services are leveraged to ensure high availability, fast response times, database backups, and SSL security.

Components and Target AWS Services:

Component / Port	Folder / Repo	AWS Target	Purpose & Access
Client App Port: 80/443 (via CDN)	lawmate-pwa	S3 + CloudFront + ACM	Builds React JS/CSS files and distributes globally via S3 & CDN under HTTPS.
API Gateway Port: 8000 (Public)	services/gateway	EC2 / ECS Fargate + ALB	Entrypoint route for all client API queries. Validates JWT and proxies requests.
Auth Service Port: 3001 (Internal)	services/auth	EC2 / ECS (Internal)	Handles user sign-in, Firebase JWT verification, and email OTP codes.
Lead Service Port: 3002 (Internal)	services/lead	EC2 / ECS (Internal)	Runs automated lawyer matching logic and SLA background timers.
Payment Service Port: 3003 (Internal)	services/payment	EC2 / ECS (Internal)	Processes Razorpay & PhonePe callback webhooks and transaction states.
Notification Service Port: 3004 (Internal)	services/notification	EC2 / ECS (Internal)	Sends push notifications (Firebase), WhatsApp alerts, and logs SMS.
Profile Service Port: 3005 (Internal)	services/profile	EC2 / ECS (Internal)	Manages user advocate profiles, ratings, and profile updates.
Database Port: 5432 (Isolated)	packages/db	AWS RDS PostgreSQL	Managed PostgreSQL database with automatic backups and failovers.

2. Identity and Access Management (IAM) Configuration

AWS IAM (Identity and Access Management) allows us to create secure, restricted access keys for deploying code to AWS without sharing our master root account. Follow these visual console clicks exactly to create a deployer user:

Step-by-Step Deployer IAM User Creation:

1. Open a new web browser tab, go to <https://aws.amazon.com/console>, and click **Sign In to the Console** in the top right corner. Select **Root User**, enter your administrator email address, enter your password, and complete any multi-factor authentication (MFA) prompts.
2. Once logged in, locate the search bar at the very top center of the console window. Type **IAM**. Click the first matching service named **IAM** (Manage access to AWS resources).
3. On the left-side navigation sidebar, locate the **Access management** drop-down menu and click on **Users**.
4. Click the orange **Create user** button located in the top-right corner of the screen.
5. ****Step 1 (Specify user details)****: In the **User name** input text field, type lawmate-deployer. Ensure the checkbox for 'Provide user access to the AWS Management Console' remains **UNCHECKED** (this account is purely for programmatic developer/CI-CD access, not web logins). Click the orange **Next** button.
6. ****Step 2 (Set permissions)****: Under ****Permissions options****, click the third card labeled **Attach policies directly**.
7. In the ****Permissions policies**** table search box, type each of the following policy names one-by-one and check the selection box on the left of each row:
 - AmazonEC2ContainerRegistryFullAccess (Allows pushing docker images to AWS ECR)
 - AmazonECS_FullAccess (Allows management of Fargate containerized services)
 - AmazonS3FullAccess (Required to write and delete client build assets in S3 buckets)
 - CloudFrontFullAccess (Allows invalidating and refreshing CDN caches globally)
8. Click the orange **Next** button at the bottom-right. Review the user details and click **Create user**.
9. On the Users list table, find and click on the blue text name lawmate-deployer that you just created.
10. Click on the **Security credentials** tab in the middle of the screen.
11. Scroll down to the section titled ****Access keys**** and click the button labeled **Create access key**.
12. Select the first radio option labeled **Command Line Interface (CLI)**. Scroll down, check the acknowledgment box confirming you understand the alternatives, and click **Next**.
13. Leave the Description tag blank, then click the orange **Create access key** button.
14. ****CRITICAL ACTION REQUIRED:**** You will see a screen displaying `Access key` and `Secret access key`. Click the gray **Download .csv file** button in the bottom left. Store this file securely; you cannot retrieve this secret key again once you leave this page.

3. Relational Database Service (RDS) Provisioning

AWS RDS handles our managed PostgreSQL instance, ensuring regular automated daily backups, automatic database OS patching, and storage scaling.

Step-by-Step RDS PostgreSQL Cluster Setup:

1. In the top console search bar, type **RDS** and select the RDS service from the results.
2. On the RDS dashboard, click the orange **Create database** button.
3. Choose a database creation method: Click the radio button for **Standard create** (allows custom network settings).
4. Engine options: Select **PostgreSQL**.
5. Engine version: Select the default stable version, **PostgreSQL 16.1 or newer**.
6. Templates: Select **Production** for live company deployment. *(For testing or staging, you may select Free Tier to minimize billing).*
7. Settings details:
 - **DB instance identifier**: Type lawoncall-prod-db.
 - **Master username**: Enter postgres.
 - **Credentials management**: Select **Self managed**. Type a strong master password in the fields and write it down in a secure credentials manager.
8. Instance configuration: Choose **Burstable classes** and select `db.t4g.medium` (2 vCPUs, 4 GiB memory, cost-effective ARM-based machine suitable for live loads).
9. Storage details:
 - **Storage type**: Select **gp3** (General Purpose SSD, offers best balance of speed and cost).
 - **Allocated storage**: Set to **20 GiB** (minimum threshold).
 - Keep **Enable storage autoscaling** checked, and set the **Maximum storage threshold** to **100 GiB**.
10. Connectivity details:
 - **Virtual Private Cloud (VPC)**: Select the default VPC or your production VPC.
 - **Public access**: Select **No**. *(This isolates the database inside the private network so hacker scanners cannot scan port 5432).*
 - **VPC security group**: Select **Create new**. Type the name lawoncall-rds-sg in the text field.
 - **Database port**: Leave set to **5432**.
11. Click the orange **Create database** button at the bottom. The database instance status will show as 'Creating'. Wait 5–10 minutes until it changes to a green **Available** status.

12. Click on the name lawoncall-prod-db. Under the **Connectivity & security** tab, copy the value under **Endpoint** (e.g. lawoncall-prod-db.xxxxxx.ap-south-1.rds.amazonaws.com). This is your database server address.

Step-by-Step Security Group Rules Config:

To allow your backend server to connect to this database, we must modify the database firewall rules:

1. In RDS console -> Click on lawoncall-prod-db. Under the **Connectivity & security** tab, look at the **VPC security groups** section. Click on the link for lawoncall-rds-sg.
2. You will be redirected to the Security Groups screen. Select the checkbox for lawoncall-rds-sg.
3. In the lower half of the screen, click the **Inbound rules** tab, then click the **Edit inbound rules** button.
4. Click **Add rule**.
5. Under **Type**, choose **PostgreSQL (Port 5432)**.
6. Under **Source**, select **Custom**. In the text field, start typing the security group of your EC2 backend server (usually named something like launch-wizard-1 or lawoncall-ec2-sg) and select it. Alternatively, type the VPC CIDR range (e.g. 172.31.0.0/16) to allow any internal server in the private subnet to connect.
7. Click the orange **Save rules** button.

SECURITY WARNING: Never open database access port 5432 to 0.0.0.0/0 (the entire internet). This exposes the database to brute force attempts. You must restrict port 5432 inbound connections exclusively to the security group assigned to your backend EC2 instance or ECS Fargate task.

3.2: Running Database Schema Migration

To initialize the tables inside the RDS PostgreSQL instance, you must execute the Prisma migration tool. This will look at the schema definition in the project files and automatically generate tables, indexes, and relations inside your blank database.

1. Open the command terminal on your local system (Command Prompt on Windows, Terminal on Mac/Linux).
2. Navigate to the folder where you have downloaded the project: `cd /path/to/Lawmate`
3. Set the database URL variable in your terminal session, substituting the database endpoint and master password you wrote down in Section 3:

```
# For Windows Command Prompt (cmd):  
set DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=require"  
  
# For Windows PowerShell:  
$env:DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=require"  
  
# For Mac or Linux Terminal:  
export DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=require"
```

4. Navigate to the database utility package: `cd packages/db`
5. Run the push command: `npx prisma db push`. Wait for the terminal to print ✓ Your database is now in sync with your Prisma schema.

4. Frontend Deployment (AWS S3 + CloudFront + ACM)

The React PWA frontend (single-page application) is built into static assets (HTML, JS, CSS) and hosted on AWS S3, while CloudFront acts as a global CDN to distribute files under HTTPS with minimum latency.

Step 4.1: Build React Assets locally:

1. Create an .env.production file inside the lawmate-pwa directory using Notepad (Windows) or TextEdit (Mac).
2. Paste your production environment configuration (Vite requires variables to be prefixed with VITE_):

```
VITE_API_URL=https://api.yourdomain.com
VITE_FIREBASE_API_KEY=AizaSyA123456789...
VITE_FIREBASE_AUTH_DOMAIN=lawmate-prod.firebaseio.com
VITE_FIREBASE_PROJECT_ID=lawmate-prod
VITE_FIREBASE_STORAGE_BUCKET=lawmate-prod.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=8877665544
VITE_FIREBASE_APP_ID=1:8877665544:web:abcd1234ef
```

3. Open your terminal, navigate to the PWA folder, and compile the static build:

```
cd lawmate-pwa
npm install
npm run build # This outputs all static files in the lawmate-pwa/dist/ folder
```

Step 4.2: Create and Configure S3 Bucket:

1. Open S3 console and click **Create bucket**.
2. Bucket Name: Enter a unique name (e.g. lawmate-pwa-prod-bucket). Select your deployment Region.
3. Under **Block Public Access settings for this bucket**, make sure **Block all public access** is **checked**. (This ensures files are served only through the CloudFront CDN securely). Click **Create bucket**.
4. Uploading assets step-by-step: Click on the name of the bucket lawmate-pwa-prod-bucket from the S3 list. Click the **Upload** button. Click the ****Add files**** button, select all files inside the local lawmate-pwa/dist/ directory, and click open. Click the ****Add folder**** button, select the assets folder inside lawmate-pwa/dist/, and click open. Click the orange ****Upload**** button at the bottom of the page and wait for the green bar.

Step 4.3: Request SSL Certificate (ACM):

To enable secure HTTPS encryption for the frontend, we must request a certificate:

1. In AWS Console search bar, type **Certificate Manager** and click on it.
2. Click ****Request certificate**** in the top right. Select ****Request a public certificate****, then click ****Next****.

3. **Fully qualified domain name**: Enter ``www.yourdomain.com``. Click **Add another name** and enter ``yourdomain.com``.
4. Select **DNS validation** as the validation method and click **Request**.
5. Refresh the certificates list, click on the ID of your request, and scroll down to the **Domains** section. Copy the CNAME Name and CNAME Value, and add them as a CNAME record at your domain provider (e.g., GoDaddy, Namecheap) to prove ownership. Once done, the status will turn to a green **Issued**.

Step 4.4: Configure CloudFront Distribution:

1. Open CloudFront console, click **Create distribution**.
2. Origin Domain: Select your S3 bucket (e.g., lawmate-pwa-prod-bucket.s3.amazonaws.com).
3. Origin Access: Select **Origin Access Control (OAC)**. Click **Create control setting**, accept defaults, and click **Create**.
4. Viewer Protocol Policy: Select **Redirect HTTP to HTTPS**.
5. Web Application Firewall (WAF): Select **Do not enable security protections** for basic setup, or enable it for DDoS guard.
6. SSL Certificate & Domain: Under **Alternate domain name (CNAME)**, enter your domain name (e.g., www.yourdomain.com). Under **Custom SSL certificate**, select your requested ACM certificate.
7. Default Root Object: Type index.html. Click **Create distribution**.
8. **CRITICAL STEP (Link S3 and CDN)**: Once created, a yellow alert bar will appear at the top saying: 'The S3 bucket policy needs to be updated'. Click the **Copy policy** button. Go to your S3 bucket's **Permissions** tab, scroll to **Bucket policy**, click **Edit**, paste the policy, and click **Save changes**.
9. **CRITICAL React Router Configuration**: In the CloudFront console under your distribution, go to the **Error pages** tab. Click **Create custom error response**. For HTTP Error Code, select **404: Not Found**. Select **Customize Error Response** -> **Yes**. Set Response Page Path to /index.html and HTTP Response Code to **200: OK**. Click **Create**.

5. Cost-Effective Backend EC2 Deployment

For cost-efficiency, ease of setup, and single-instance management, we deploy the Fastify Gateway API and microservices on a single AWS EC2 instance orchestrated via Docker Compose and served behind an Nginx reverse proxy with SSL.

Step 5.1: Provision EC2 Instance:

1. Open the EC2 dashboard, click **Launch instance**.
2. Instance name: lawmate-backend-server. OS: Select **Ubuntu Server 22.04 LTS**.
3. Instance Type: Select t3.medium (2 vCPUs, 4 GiB memory) to handle running all microservices smoothly.
4. Key Pair: Select an existing key pair or click **Create new key pair**. Download the .pem file and keep it secure on your system.
5. Network Settings: Check the boxes for **Allow SSH traffic from**, **Allow HTTPS traffic from the internet**, and **Allow HTTP traffic from the internet**.
6. Click **Launch instance**.
7. **Allocate Elastic IP**: On the EC2 sidebar, select **Elastic IPs**, click **Allocate Elastic IP address**. Select the allocated IP, click **Actions** -> **Associate Elastic IP address**, and select your running lawmate-backend-server instance. (This ensures the IP address stays constant after restarts).

Step 5.2: Connect to the Server & Install Software:

1. In the EC2 instances list, select lawmate-backend-server and click the **Connect** button at the top.
2. Under the **EC2 Instance Connect** tab, click the orange **Connect** button. (This opens a secure server command line terminal directly in your web browser, removing the need for any SSH client software!).
3. Once the command prompt appears, copy and paste the following commands to install Node.js 20, Docker, Docker Compose, Git, Nginx, and Certbot:

```
# Setup nodesource Node.js repository and install required tools
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get update
sudo apt-get install -y nodejs docker.io docker-compose git certbot python3-certbot-nginx
```

Step 5.3: Set Up Project Files & Dependencies:

1. Create the application folder under /app on the EC2 server and grant ownership permissions to the current user (ubuntu):

```
sudo mkdir -p /app
```

```
sudo chown -R ubuntu:ubuntu /app
```

2. Clone the Git repository of the project into the /app/lawmate directory:

```
git clone https://github.com/codeamanwal/Lawmate.git /app/lawmate  
cd /app/lawmate
```

3. Create the .env environment file inside the project directory: nano .env. (Paste the configuration checklist from Section 6, replacing placeholders with your actual secrets). Paste using right-click in browser terminal. Press Ctrl+O to save, press Enter to confirm file name, and then press Ctrl+X to exit the nano editor.
4. Install all project dependencies recursively for all microservices (this handles standard compilation for the host OS environment):

```
npm run install:all
```

5. Generate the database Prisma Client models and push the database schema directly to your RDS instance:

```
npm run db:generate  
npm run db:push
```

6. Create the docker-compose.yml file: nano docker-compose.yml. Paste the configuration template shown on the next page, then press Ctrl+O to save, press Enter to confirm file name, and then press Ctrl+X to exit.
7. Start all backend microservices using Docker Compose: sudo docker-compose up -d. (This downloads Node.js, mounts the local files, and launches all microservices in the background). Verify status with: sudo docker-compose ps.

5.4: docker-compose.yml Production Template

This configuration launches all the LawMate microservices on the EC2 instance, using `network_mode: 'host'` for optimal performance and cross-service communication.

```
version: '3.8'

services:
  gateway:
    image: node:20
    working_dir: /app
    volumes:
      - ../app
    command: npx tsx services/gateway/index.ts
    network_mode: "host"
    environment:
      - PORT=8000
      - JWT_SECRET=${JWT_SECRET}
      - FRONTEND_URL=${FRONTEND_URL}
      - AUTH_SERVICE=http://127.0.0.1:3001
      - LEAD_SERVICE=http://127.0.0.1:3002
      - PROFILE_SERVICE=http://127.0.0.1:3005
      - PAYMENT_SERVICE=http://127.0.0.1:3003
      - NOTIFICATION_SERVICE=http://127.0.0.1:3004
    restart: always

  auth:
    image: node:20
    working_dir: /app
    volumes:
      - ../app
    command: npx tsx services/auth/index.ts
    network_mode: "host"
    environment:
      - PORT=3001
      - DATABASE_URL=${DATABASE_URL}
      - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}
      - FIREBASE_CLIENT_EMAIL=${FIREBASE_CLIENT_EMAIL}
      - FIREBASE_PRIVATE_KEY=${FIREBASE_PRIVATE_KEY}
      - JWT_SECRET=${JWT_SECRET}
      - SMTP_HOST=${SMTP_HOST}
```

```
- SMTP_PORT=${SMTP_PORT}
- SMTP_USER=${SMTP_USER}
- SMTP_PASS=${SMTP_PASS}
restart: always

lead:
  image: node:20
  working_dir: /app
  volumes:
    - ./app
  command: npx tsx services/lead/index.ts
  network_mode: "host"
  environment:
    - PORT=3002
    - DATABASE_URL=${DATABASE_URL}
    - EXOTEL_API_KEY=${EXOTEL_API_KEY}
    - EXOTEL_API_TOKEN=${EXOTEL_API_TOKEN}
    - EXOTEL_ACCOUNT_SID=${EXOTEL_ACCOUNT_SID}
    - EXOTEL_SUBDOMAIN=${EXOTEL_SUBDOMAIN}
    - EXOTEL_EXOPHONE=${EXOTEL_EXOPHONE}
    - EXOTEL_STATUS_CALLBACK_URL=${EXOTEL_STATUS_CALLBACK_URL}
  restart: always

payment:
  image: node:20
  working_dir: /app
  volumes:
    - ./app
  command: npx tsx services/payment/index.ts
  network_mode: "host"
  environment:
    - PORT=3003
    - DATABASE_URL=${DATABASE_URL}
    - PHONEPE_CLIENT_ID=${PHONEPE_CLIENT_ID}
    - PHONEPE_CLIENT_SECRET=${PHONEPE_CLIENT_SECRET}
    - PHONEPE_CLIENT_VERSION=${PHONEPE_CLIENT_VERSION}
    - PHONEPE_ENV=${PHONEPE_ENV}
    - PHONEPE_WEBHOOK_USERNAME=${PHONEPE_WEBHOOK_USERNAME}
    - PHONEPE_WEBHOOK_PASSWORD=${PHONEPE_WEBHOOK_PASSWORD}
    - FRONTEND_URL=${FRONTEND_URL}
    - JWT_SECRET=${JWT_SECRET}
```

```
restart: always

notification:
  image: node:20
  working_dir: /app
  volumes:
    - ./app
  command: npx tsx services/notification/index.ts
  network_mode: "host"
  environment:
    - PORT=3004
    - DATABASE_URL=${DATABASE_URL}
    - FIREBASE_PROJECT_ID=${FIREBASE_PROJECT_ID}
    - FIREBASE_CLIENT_EMAIL=${FIREBASE_CLIENT_EMAIL}
    - FIREBASE_PRIVATE_KEY=${FIREBASE_PRIVATE_KEY}
  restart: always

profile:
  image: node:20
  working_dir: /app
  volumes:
    - ./app
  command: npx tsx services/profile/index.ts
  network_mode: "host"
  environment:
    - PORT=3005
    - DATABASE_URL=${DATABASE_URL}
  restart: always
```

5.5: Nginx Web Server SSL Setup (Certbot)

Nginx acts as a reverse proxy. It listens on public ports 80 (HTTP) and 443 (HTTPS), redirects all HTTP web requests to HTTPS, and forwards API queries targeting /api directly to the local Gateway running on port 8000.

Step 5.5.1: Create Nginx Site Configuration:

1. Open the Nginx config editor: `sudo nano /etc/nginx/sites-available/lawoncall`
2. Paste the following configuration, replacing `api.yourdomain.com` with your actual backend subdomain:

```
server {  
    listen 80;  
    server_name api.yourdomain.com;  
  
    location / {  
        proxy_pass http://localhost:8000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_cache_bypass $http_upgrade;  
    }  
}
```

3. Enable the configuration by linking it to the active sites folder:

```
sudo ln -s /etc/nginx/sites-available/lawoncall /etc/nginx/sites-enabled/  
sudo rm /etc/nginx/sites-enabled/default # Remove default page config  
sudo nginx -t # Test if syntax is correct  
sudo systemctl restart nginx # Apply changes
```

Step 5.5.2: Install Free SSL Certificate via Let's Encrypt:

To enable secure HTTPS encryption, run Certbot to automatically request and install your SSL certificate. Certbot will also automatically update Nginx settings for you.

```
sudo certbot --nginx -d api.yourdomain.com
```

Follow the prompts on-screen: Enter your email, agree to the terms, and choose to **Redirect all HTTP traffic to HTTPS** automatically.

6. Environment Variables & Credentials Checklist

Make sure your production environment variables inside the `/app/lawmate/.env` file on the EC2 instance are populated with the correct API keys. Below is the checklist of required values:

Variable Key	Category	Value Description / Production Placeholder
DATABASE_URL	Database	postgresql://postgres:YOUR_PASSWORD@rds-endpoint.amazonaws.com:5432/neondb?sslmode=require
JWT_SECRET	Security	Any cryptographically secure random string used to sign user auth tokens.
FRONTEND_URL	Security	The URL of your frontend CloudFront CDN (e.g. <code>https://www.yourdomain.com</code>).
FIREBASE_PROJECT_ID	Auth	Your Firebase Console project ID (e.g. <code>lawmate-prod</code>).
FIREBASE_CLIENT_EMAIL	Auth	Firebase Service Account client email address.
FIREBASE_PRIVATE_KEY	Auth	CRITICAL FORMATTING: Copy the service account private key from the JSON file. Paste it enclosed in double quotes (e.g. <code>"-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"</code>). The backslash-n characters are resolved programmatically at runtime.
PHONEPE_CLIENT_ID	Payment	Your PhonePe API Merchant Client ID.
PHONEPE_CLIENT_SECRET	Payment	Your PhonePe API Client Secret key.
PHONEPE_CLIENT_VERSION	Payment	PhonePe API integration Client Version (defaults to 1).
PHONEPE_ENV	Payment	Set to <code>PRODUCTION</code> for live payment checkouts, or <code>SANDBOX</code> for testing.
PHONEPE_WEBHOOK_USERNAME	Payment	Basic Auth username for securing PhonePe webhooks (defaults to <code>admin</code>).
PHONEPE_WEBHOOK_PASSWORD	Payment	Basic Auth password for securing PhonePe webhooks (defaults to <code>password123</code>).
EXOTEL_API_KEY	Calls/Exotel	Your Exotel account API key.
EXOTEL_API_TOKEN	Calls/Exotel	Your Exotel account API token.
EXOTEL_ACCOUNT_SID	Calls/Exotel	Your Exotel Account SID.
EXOTEL_SUBDOMAIN	Calls/Exotel	Exotel API subdomain (typically <code>api.exotel.com</code>).
EXOTEL_EXOPHONE	Calls/Exotel	Your Exotel virtual ExoPhone phone number.

Variable Key	Category	Value Description / Production Placeholder
EXOTEL_STATUS_CALLBACK_URL	Calls/Exotel	Your backend call callback URL (e.g. https://api.yourdomain.com/api/leads/call-status).
SMTP_HOST	Email	Mail server domain (e.g. smtp.gmail.com).
SMTP_PORT	Email	Mail port (typically 587 for TLS/STARTTLS).
SMTP_USER	Email	Email account username (e.g. support@yourdomain.com).
SMTP_PASS	Email	Email account password (use a Gmail App Password if using Gmail).

7. Troubleshooting & Verification Checklist

Use these common administrative commands on your EC2 instance terminal to manage the services and troubleshoot errors:

Action Needed	Exact Command to Run
Check microservice logs in real-time	<code>sudo docker-compose logs -f [service_name]</code> (e.g., <code>sudo docker-compose logs -f gateway</code>)
Check logs for all backend services	<code>sudo docker-compose logs --tail=100 -f</code>
Restart all backend microservices	<code>sudo docker-compose restart</code>
Apply environment variable changes	<code>sudo docker-compose down && sudo docker-compose up -d</code>
Check server memory and disk space usage	<code>df -h && free -h</code>
Check active Docker containers and resource usage	<code>sudo docker ps && sudo docker stats</code>
Check if microservice ports are binding correctly	<code>sudo ss -tuln grep -E '8000 3001 3002 3003 3004 3005'</code>
Test backend API Gateway health endpoint	<code>curl -i http://localhost:8000/api/health</code>
Test microservices direct status checks	<code>curl -i http://localhost:3001/ && curl -i http://localhost:3002/</code>

8. Verification Checklist for Go-Live:

- **DNS Mapping:** Point ``api.yourdomain.com`` (backend) to the EC2 Elastic IP using an A Record in your registrar console (e.g. Route53, GoDaddy).
- **CloudFront SSL:** CNAME ``www.yourdomain.com`` points to the CloudFront distribution domain (``xxxx.cloudfront.net``).
- **Database Connectivity:** Confirm backend services show successful PostgreSQL connection logs.
- **Let's Encrypt SSL Auto-renewal:** Certbot handles renewals automatically, check cron status using ``sudo systemctl status certbot.timer``.

End of Documentation.